



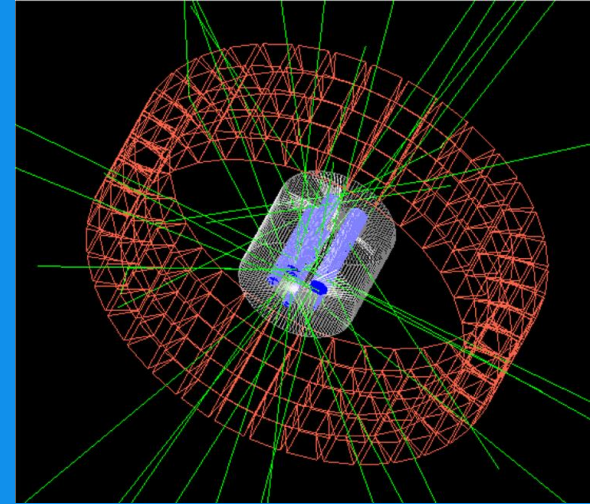
Tecniche Sperimentali per la Fisica delle Alte Energie

Introduzione a Geant4

S. Germani – stefano.germani@unipg.it



Geant4 è un toolkit open source che consente di simulare il passaggio delle particelle attraverso la materia. È scritto in C++ e offre un'ampia gamma di strumenti che gli utenti possono utilizzare nelle loro applicazioni. *(traduzione della homepage di Geant4)*



- Geant4 è un toolkit:
 - Non fornisce automaticamente un programma da eseguire;
 - L'utente deve scrivere un'applicazione che usi gli strumenti (tool) forniti da Geant4;
 - Si inizia in genere a partire dagli esempi forniti.

Concetti Base

Particella Primaria

Descrizione Rivelatore

- Volumi
- Materiali
- Zone Sensibili

Tracciamento (Tracking)

- Prende in carico ogni particella dalla generazione allo stop

Step

- Il trasporto di ogni particella e l'interazione con il materiale avviene per passi (step)
- Ad ogni Step si ha in genere un solo tipo di processo discreto (perdita per ionizzazione ed emissione Cerenkov sono trattati come processi continui)

Hit

- Per ogni Step l'interazione della traccia nella regione sensibile da origine ad un Hit
- Le informazioni immagazzinate per ogni Hit devono essere definite dall'utente

Materiali

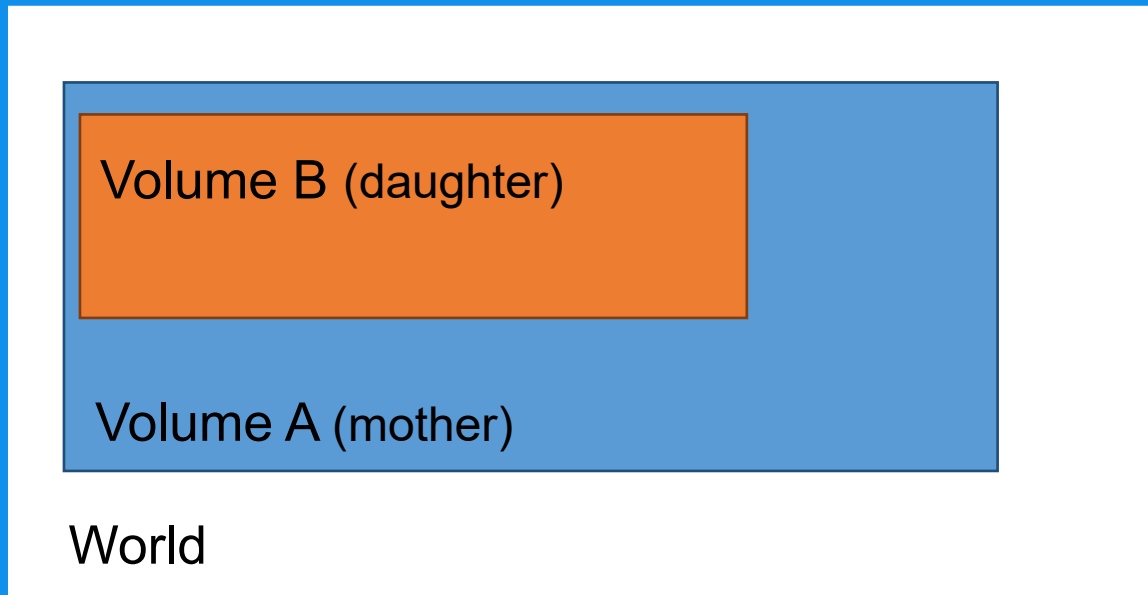
Definizione Elementi

```
G4double a = 1.01*g/mole;  
G4Element* H = new G4Element(name="Hydrogen",symbol="H" , z= 1., a);  
  
a = 12.01*g/mole;  
G4Element* C = new G4Element(name="Carbon", symbol="C", z=6., a);
```

Definizione Materiali

```
G4double density = 1.032*g/cm3;  
G4Material* scintillator = new G4Material(name = "Scintillator", density, numberOfComponents =  
2);  
  
scintillator -> AddElement(C, numberOfAtoms = 9);  
scintillator -> AddElement(H, numberOfAtoms = 10);
```

Geometria Rivelatore



Il volume A è madre del Volume B

Il volume figlio deve essere contenuto interamente nel volume madre

Evitare sovrapposizioni e protrusioni è uno dei problemi principali nella costruzione della geometria del rivelatore

Ogni Volume è costituito da tre livelli concettuali:

- Solido (*G4Solid*): forma, dimensione;
- Volume Logico (*G4LogicalVolume*): materiale, sensibilità, campo magnetico, ...
- Volume Fisico (*G4Logical Volume*): posizione nello spazio.

Geometria Rivelatore

G4Solid

- Esistono diversi tipi di solido che possono anche essere raggruppati.
- Esiste anche un'interfaccia CAD
- E' bene verificare i metodi possibili caso per caso sulla base delle necessità

G4LogicalVolume

- Una volta decise le proprietà del volume l'implementazione è univoca

G4PhysicaVolume

- Un volume fisico può avere una corrispondenza biunivoca con un volume logico
- Allo stesso volume logico possono corrispondere più volumi fisici tramite le repliche
 - L'identificazione del volume fisico rimane univoca tramite il *Replica Number*

Run / Event

Un Run (*G4Run*) è un insieme di eventi simulati in cui le condizioni del rivelatore rimangono le stesse.

Un Run inizia con un «Beam On»

Un Evento (*G4Event*) corrisponde alla generazione di una particella primaria e allo sviluppo di tutte le reazioni che ne conseguono.

Alla fine di un evento si possono avere a disposizione informazioni su:

- Particelle e vertici primari
- Collezione di Hit e Traiettorie

Track

Un traccia (*G4Track*) corrisponde ad una istantanea di una particelle lungo lo sviluppo della simulazione con una posizione e delle quantità fisiche associate.

Una traccia è considerata morta se:

- Esce dal volume World
- Sparisce per via interazioni o decadimenti
- Ha perso tutta l'energia cinetica

In Geant4 le particelle sono trattate in maniera classica cioè corrispondono ad un oggetto puntiforme con quantità di moto e posizione definite (no funzione d'onda). Le probabilità di interazione tengono però conto di effetti quantistici.

G4Trajectory è la classe che mantiene persistenti le informazioni di *G4Track* alla fine dell'evento.

Step

Lo sviluppo della simulazione di una traccia procede per passi discreti (Step)

Ad ogni passo i vari processi (incluso il trasporto) concorrono probabilisticamente a determinare la lunghezza dello step e il tipo di interazione nello step stesso. Le interazioni di uno step possono dare origine a particelle secondarie.

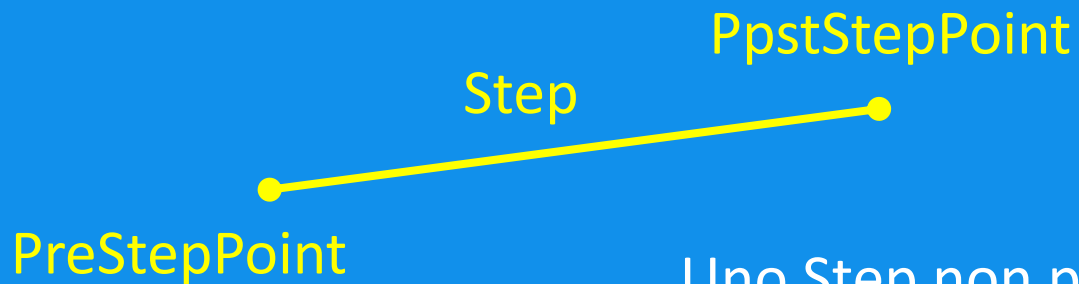
Uno Step (*G4Step*) corrisponde all'informazione differenziale della traccia con

- Posizione iniziale e finale
- Processo;
- Energia iniziale e finale della traccia
- Energia depositata.

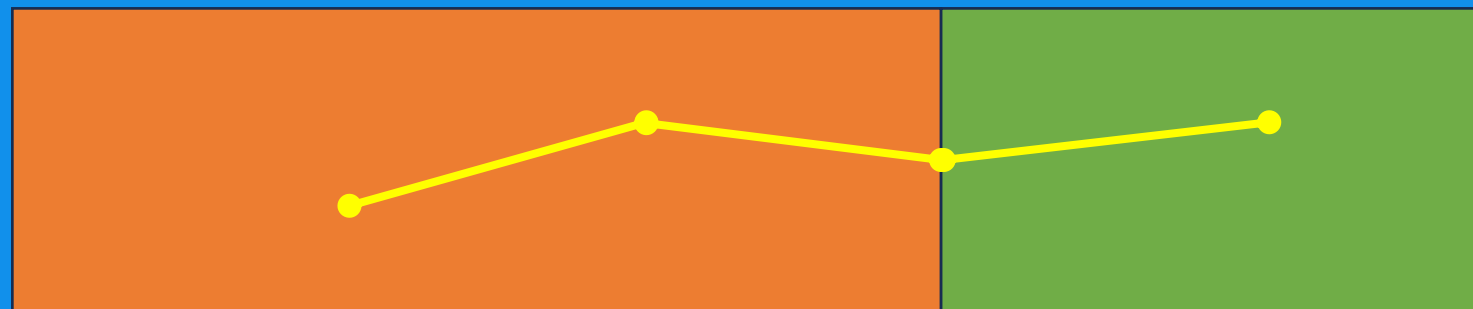
G4TrajectoryPoint è la classe che mantiene persistenti le informazioni di *G4Step* alla fine dell'evento.

Step

- Ad ogni particella corrisponde una lista di processi applicabili;
- Ad ogni Step tutti i processi sono invocati per ottenere probabilisticamente una lunghezza di interazione fisica
- Il processo che richiede la lunghezza di interazione minore limita la lunghezza dello step



Uno Step non può attraversare il confine fra due volumi



Production Cut

Può essere definita una soglia di produzione per le particelle di produzione sotto forma di range minimo (Production Cut) poi tradotta in una soglia di energia che dipende dal materiale.

Se una particella non viene prodotta poiché non supera il Production Cut la sua energia cinetica contribuisce al deposito di energia continuo lungo lo Step

Risposta del Rivelatore

ESEMPI:

Posizione Hit tracciatore

Energia depositata in un Calorimetro

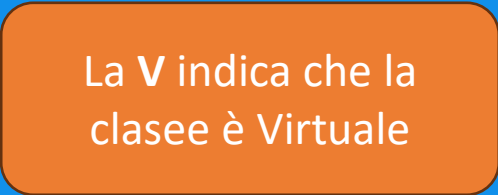
Distribuzione Hit rivelatore Cerenkov

L'utente deve fornire la propria implementazione della risposta del rivelatore

- Parti del rivelatore sensibili (Sensitive Detector)
- Hit nel rivelatore
- Geometria di lettura
- Digitizzazione

Sensitive Detector

- Un Volume Logico può essere reso Sensibile
 - Attraverso un puntatore ad un classe derivata da *G4VSensitiveDetector*
- Un volume sensibile è legato alla creazione o modifica di Hit (*G4Hit*) attraverso le informazioni disponibili in uno step tramite *G4Step*.



La V indica che la classe è Virtuale

Hit

La **V** indica che la classe è Virtuale

Il concetto di Hit è legato ad una classe definita dall'utente derivata da *G4VHit*

Implementando la propria classe Hit concreta si possono immagazzinare le informazioni disponibili a livello dello step, ad esempio:

- Posizione e Tempo dello step;
- Energia e Quantità di Moto della Traccia;
- Energia Depositata nello step
- Informazioni geometriche
- ...

Per ogni Evento le informazioni degli Hit vengono immagazzinate in una «collection» (*G4HCofThisEvent*)

Digitizzazione

La rappresentazione della risposta dell'elettronica di acquisizione è legata al concetto di Digi

- ACD / TDC, trigger ...

Un Digi raccoglie informazioni da uno o più Hit o altri Digi

Informazioni Fornite dall'Utente

Cosa l'UTENTE DEVE fare:

- Descrivere il setup sperimentale
- Fornire le particelle primarie della simulazione
- Decidere modelli fisici da utilizzare nella simulazione e la condizione (cut) per arrestare il tracciamento di una particella.

Classi User

Geant4 mette a disposizione le classi User come strumenti per configurare, controllare e gestire la simulazione.

Classi di Inizializzazione

- G4VUserDetectorConstruction
- G4VUserPhysicsList

Classi di Azione

- *G4VUserPrimaryGeneratorAction*
- *G4UserRunAction*
- *G4UserEventAction*
- *G4UserTrackingAction*
- *G4UserStackingAction*
- *G4UserSteppingAction*

Classi Obbligatorie

Classi Opzionali User Action

- G4User**Run**Action
- G4User**Event**Action
- G4User**Tracking**Action
- G4User**Stacking**Action
- G4User**Stepping**Action

Ogni funzione delle classi base ha un'implementazione dummy che non fa nulla

L'utente può implementare tutte le funzioni che vuole nella propria classe derivata

L'oggetto creato va registrato con il *G4RunManager*

Classi Opzionali User Action

G4UserRunAction

- BeginOfRunAction(const G4Run*)
- EndOfRunAction(const G4Run*)

G4UserTrackingAction

- PreUserTrackingAction(const G4Track*)
- PostUserTrackingAction(const G4Track*)

G4UserStackingAction

- PrepareNewEvent()
- ClassifyNewTrack(const G4Track*)
- NewStage()

G4UserEventAction

- BeginOfEventAction(const G4Event*)
- EndOfEventAction(const G4Event*)

G4UserSteppingAction

- UserSteppingAction(const G4Step*)

main()

Geant4 non fornisce un *main()* che è parte dell'applicazione realizzata dall'utente

Il *main()* deve contenere:

- Il costruttore della classe *G4RunManager*
- La notifica al *G4RunManager* riguardo alle classi derivate dell'utente:
 - *G4VUserDetectorConstruction*
 - *G4VUserPhysicsList*
 - *G4VUserPrimaryGeneratorAction*

Il *main()* può contenere anche la definizione delle classi utente opzionali

Genat4 UI

Geant4 fornisce una interfaccia utente (UI) con la propria applicazione per controllare e configurare la simulazione basata su comandi.

Un comando può essere dato da:

- Interfaccia UI
- File Macro
- Direttamente nel codice (hardcoded)

Struttura comando:

- Command directory (area concettuale che riguarda il comando)
- Comando
- Parametro/i

```
G4UImanager* UI = G4UImanager::GetUIpointer();  
UI->ApplyCommand("/run/verbose 1");
```

Macro

Una macro è un file ASCII contenente una serie di comandi UI

```
/control/verbose 2  
/control/saveHistory  
/run/verbose 2  
  
/gun/particle gamma  
/gun/energy 1 GeV  
/gun/sourceType 2  
  
# run 10 events  
/run/beamOn 10
```

Ricetta base per l'utente (principiante)

- Creare le classi utenti obbligatorie
 - MyDetectorConstruction
 - MyPhysicsList
 - MyPrimaryGeneratorAction
- Se necessario creare le classi utente di azione opzionali
 - MyUserRunAction
 - MyUserEventAction
 - MyUserTrackingAction
 - MyUserStackingAction
 - MyUserSteppingAction
- Creare il *main()*
 - Istanziando G4RunManager o la classe derivata MyRunManager
 - Notificare il RunManager riguardo alle classi utente obbligatorie e opzionali
 - Opzionalmente istanziare l'interfaccia utente e di visualizzazione
- Creare Macro di configurazione